Employee Management System

with AI-based Analytics | Java + JDBC + MySQL

Tasks · AI Analytics · Console Menus

Jothish Kumar P (251801123)

B.Tech Artificial Intelligence and Data Science | Rajalakshmi Engineering College (Autonomous)

Guide: Mr. T. Rajesh, M.Tech., Assistant Professor (SS) | Team: Jai Mukeshwaran V, Jaques Kevin R, Jothish Kumar P

Individual presentation: explanation · purpose · uses · advantages · basic needs · code walkthrough · execution

Project at a glance

Problem: Institutions still keep employee records, attendance, leave and salary in registers and spreadsheets → errors, delays, unfair workload and no performance view.

Solution: a Java console application with OOP, JDBC and MySQL that automates the employee cycle and adds explainable AI analytics.

Jai Mukeshwaran V (251801107)
Database design · DBConnection · EmployeeService · Login

Jaques Kevin R (251801109)
Attendance · Leave · Payroll

Jothish Kumar P (251801123)
Tasks · AI Analytics · Console menus · Testing

What I built

- **TaskService** – assign tasks with hours, due date and priority; employees update status.
- **AnalyticsService + AnalyticsRules** – performance score, workload analysis, task suggestion, attendance-risk alerts.
- **Main (console)** – Admin and Employee menus, input validation (readInt, readMoney, readDate), plain-language messages.
- **Testing** – black-box test cases and the LogicTest-style automated checks.

Purpose and real-world uses

Purpose

Turn stored data into decisions: who performs well, who is overloaded, who should get the next task, who is at attendance risk.

Uses in the real world

Workload balancing for faculty, hospital staff or project teams; early warning for absenteeism; fair, explainable performance review for appraisals.

- **Explainable AI first:** every score traces back to stored data – important for fair HR decisions.
- **Upgrade path:** attrition prediction, HR chatbot, resume screening, smart scheduling, anomaly detection (including suspected proxy attendance) and payroll forecasting.

Advantages and basic needs

Module-level benefits

Advantages

- Objective, repeatable performance score
- Workload spread fairly – fewer burn-outs
- Task suggestion saves the Admin's time
- Early warning on attendance risk
- Simple rules – easy to explain in a viva or to staff

Basic needs covered

- Task assignment and tracking
- Reports: performance, workload, risk
- Role-specific menus (13 Admin, 10 Employee options)
- Lists with numbers before choices
- Validated input and clear messages

AI analytics: how the score works

Weighted performance score

Overall = 0.4·A + 0.4·T + 0.2·P

A = attendance % · T = tasks done on time % · P = punctuality %
Example: A=92, T=80, P=90 → 36.8 + 32 + 18 = **86.8 (Excellent)**

Ratings

85+ Excellent · 70–84 Good · 50–69 Average · below 50 Needs Improvement

Workload

Utilisation = pending hours ÷ weekly limit × 100 (20 h teaching, 30 h non-teaching). >100% OVERLOADED · 40–100% BALANCED · <40% UNDERLOADED

Attendance risk

LATE + ABSENT this month: 5+ HIGH · 3–4 MEDIUM · <3 LOW

Code walkthrough 1: AI rules


```
static double overall(double a, double t, double p) { return 0.4*a + 0.4*t + 0.2*p; }
static String rating(double s) {
    return s >= 85 ? "Excellent" : s >= 70 ? "Good" : s >= 50 ? "Average"
                                                            : "Needs Improvement";
}
static String workload(double pendingHrs, int maxHrs) {
    double u = pendingHrs / maxHrs * 100;
    return u > 100 ? "OVERLOADED" : u >= 40 ? "BALANCED" : "UNDERLOADED";
}
static String risk(int lateAbsent) {
    return lateAbsent >= 5 ? "HIGH" : lateAbsent >= 3 ? "MEDIUM" : "LOW";
}
```

```
// Greedy least-loaded heuristic
candidates.sort(Comparator
    .comparingDouble((Candidate c) -> c.loadAfter(newHours))   // lowest load first
    .thenComparing(c -> -c.performanceScore));                 // tie -> higher score
Candidate best = candidates.get(0);
```

Greedy least-loaded

For a new task, compute each ACTIVE, available employee's load after adding the hours. Pick the lowest; on a tie choose the higher performance score. The Admin can accept or override.

Code walkthrough 2: console input validation

Main.java

```
static int readInt(Scanner in, String prompt) {
    while (true) {
        System.out.print(prompt);
        try { return Integer.parseInt(in.nextLine().trim()); }
        catch (NumberFormatException e) { System.out.println("Please enter a whole number."); }
    }
}
static LocalDate readDate(Scanner in, String prompt) {
    while (true) {
        System.out.print(prompt);
        try { return LocalDate.parse(in.nextLine().trim()); }
        catch (DateTimeParseException e) {
            System.out.println("Please use the format YYYY-MM-DD, e.g. 2026-11-05.");
        }
    }
}
```

Why validate?

Menu programs fail when users type text instead of numbers or wrong date formats. The helpers keep asking until a valid value is entered, with a friendly message – never a stack trace.

Execution: real output of the AI rules

Run: java EmsLogic.java (JDK 21)

```
[PASS] Overall A=92,T=80,P=90  expected=86.80  actual=86.80
[PASS] Attendance score (18,2,2,2)  expected=91.67  actual=91.67
[PASS] Task score 4 of 5  expected=80.00  actual=80.00
[PASS] Punctuality 18 present 2 late  expected=90.00  actual=90.00
[PASS] No attendance recorded  expected=0.00  actual=0.00
[PASS] No task due  expected=100.00  actual=100.00
[PASS] Rating 85  expected=Excellent  actual=Excellent
[PASS] Rating 84.99  expected=Good  actual=Good
[PASS] Rating 70  expected=Good  actual=Good
[PASS] Rating 69.99  expected=Average  actual=Average
[PASS] Rating 50  expected=Average  actual=Average
[PASS] Rating 49.99  expected=Needs Improvement  actual=Needs Improvement

=== 4. Workload analysis ===
[PASS] Teaching 24 h pending  expected=OVERLOADED  actual=OVERLOADED
[PASS] Non-teaching 15 h pending  expected=BALANCED  actual=BALANCED
[PASS] Teaching 6 h pending  expected=UNDERLOADED  actual=UNDERLOADED
[PASS] Exactly 100%  expected=BALANCED  actual=BALANCED
[PASS] Exactly 40%  expected=BALANCED  actual=BALANCED

=== 5. Attendance risk ===
[PASS] 2 late/absent  expected=LOW  actual=LOW
```

```
[PASS] 3 late/absent  expected=MEDIUM  actual=MEDIUM
[PASS] 4 late/absent  expected=MEDIUM  actual=MEDIUM
[PASS] 5 late/absent  expected=HIGH  actual=HIGH

=== 6. Task-assignment suggestion (greedy least-loaded) ===
Rank 1: Divya M  load after = 45.00%  score = 90.00
Rank 2: Karthik P  load after = 45.00%  score = 80.00
Rank 3: Ravi S  load after = 80.00%  score = 70.00
[PASS] Best candidate (tie by score)  expected=Divya M  actual=Divya M
[PASS] Last candidate  expected=Ravi S  actual=Ravi S

=== 7. Leave and attendance rules ===
[PASS] Days 5th to 5th  expected=1  actual=1
[PASS] Days 5th to 9th  expected=5  actual=5
[PASS] Check-in 08:50  expected=PRESENT  actual=PRESENT
[PASS] Check-in 09:15  expected=PRESENT  actual=PRESENT
[PASS] Check-in 09:16  expected=LATE  actual=LATE
[PASS] Overlap: 5-9 vs 8-12  expected=true  actual=true
[PASS] Overlap: 5-9 vs 10-12  expected=false  actual=false

Result: 41 passed, 0 failed.
```

Full run: **41 passed, 0 failed**. Menu-driven tests (TC20–TC25) run on MySQL; screenshots are inserted in Appendix B from the real application.

Likely viva questions

Be ready to answer

Q. Why rule-based AI and not machine learning?

No historical data yet, and HR decisions need to be explainable; thresholds can be traced to stored data. ML is planned once data exists.

Q. What is the greedy heuristic?

Choose the locally best option – the lowest load after adding the task – with ties broken by performance score.

Q. What if an employee is on leave on the due date?

They are skipped and the Admin is told how many were skipped.

Q. How can the weights be changed?

They are constants (0.4/0.4/0.2) in AnalyticsRules; set them per college policy.

Q. Name two AI upgrades.

Attrition prediction and anomaly detection (e.g. suspected proxy attendance), with bias checks and human review.

Summary

- Four explainable analytics: score, workload, suggestion, risk
- Menus with validated input and clear messages
- Logic verified by a real run – 41 checks passed
- Next: ML models, chatbot, smart scheduling, web / mobile interface